Open Authorization: Protocol Design and Formal Analysis with OFMC

Kristófer Birgir Hjörleifsson - s253155 Mikael Máni Eyfeld Clarke - s253024

Hand-in: Mar 16, 2026

Abstract

We design and incrementally refine an open authorization protocol in which a user A delegates read-only access to a photo resource at server B to a photo book service P, using an honest identity provider `idp` (modeled as a lowercase constant to prevent the intruder from impersonating it). The protocol is specified in AnB and analyzed using OFMC under the Dolev–Yao attacker model. This report documents the development process, discovered attacks, design iterations, and the final protocol, including weekly special tasks.

Contents

1 Group Setup and Section Ownership **3**

2 Resources Used **3**

3 Scenario and Security Goals **3**
3.1 Scenario . . . 3
3.2 Agents . . . 3
3.3 Security and Functional Goals . . . 4

4 Week 2: Informal Design and Static Dolev–Yao Analysis **4**
4.1 Informal Outline . . . 4
4.2 First AnB Model (Simplified) . . . 4
4.3 OFMC Result . . . 4
4.4 Special Task: Static Analysis (Passive Intruder) . . . 5

5 Week 3: Identity Provider and Lazy Intruder Executability **5**
5.1 Protocol Refinement . . . 5
5.2 Special Task: Lazy Intruder Method (P Executable by Intruder) . . . 5

6 Week 4: Typing, Formats, and Public-Key Lookup **6**
6.1 Type-Flaw Resistance via Format Tags . . . 6
6.2 Public-Key Lookup Subprotocol . . . 6
6.3 Special Task: Type-Flaw Resistance Proof . . . 6

7 Week 5: Channels (TLS-like Pseudonymous Channels) **7**
7.1 Channel Notation . . . 7
7.2 Pseudonymous Channel Variant . . . 7
7.3 Special Task: What Was Replaced . . . 7

8 Week 6: Privacy and Guessable Password **7**
8.1 Guessable Secrets . . . 7
8.2 Security with Guessable Password . . . 8
8.3 Special Task: Insecure Variant Demonstration . . . 8

9 Final Protocol (AnB Specification Summary) **8**
9.1 Final Message Flow . . . 8
9.2 Initial Knowledge . . . 9
9.3 Goals . . . 9
9.4 Verification Results . . . 9

10 Assumptions, Simplifications, and Limitations **9**
10.1 Assumptions . . . 9
10.2 Simplifications . . . 10
10.3 Limitations . . . 10

11 Conclusion **10**

A Submission Map (Where to Find Things) **11**

1 Group Setup and Section Ownership

Section author: Kristófer Birgir Hjörleifsson

- **Group members:** Kristófer Birgir Hjörleifsson, Mikael Máni Eyfeld Clarke
- **Repository:** (https://github.com/kristoferbirgir/Project1-LogicForSecurity)
- **Submitted artifacts:** This report (PDF), AnB files, and the lab logbook.

Section ownership requirement (DTU): Each section in this report lists exactly one author. Any section without a single designated author may be counted as not submitted.

2 Resources Used

Section author: Kristófer Birgir Hjörleifsson

- Course slides and lecture notes (weeks 2–7).
- OFMC / AnB documentation and examples from the course material.
- AVISPA project documentation for AnB syntax reference.
- AI tools: GitHub Copilot, Claude (Anthropic), and ChatGPT (OpenAI) used for protocol development iteration and documentation.
- Group discussions between team members.

3 Scenario and Security Goals

Section author: Mikael Máni Eyfeld Clarke

3.1 Scenario

A stores holiday photos at server B. A wants to create a photo book using service P and wishes to authorize P to access a subset of A's photos at B without downloading and re-uploading them. A authenticates using an identity provider `idp` via a shared password `pw(A,idp)`. The password is considered guessable and must not be used as an encryption key. All parties except A have public/private keys. All parties know `idp`'s public key, and `idp` knows all public keys. In the AnB model, `idp` is written in lowercase to declare it as an honest constant—the intruder cannot instantiate this role.

3.2 Agents

- **A (User):** Owns photos, authenticates via password, no cryptographic keys.
- **B (Photo Server):** Stores A's photos, has key pair `pk(B)/inv(pk(B))`.
- **P (Printing Service):** Wants to access A's photos, has key pair `pk(P)/inv(pk(P))`.
- **idp (Identity Provider):** Authenticates A, issues authorization tokens, has key pair `pk(idp)/inv(pk(idp))`. Modeled as a lowercase constant (honest, trusted).

3.3 Security and Functional Goals

We target the following goals:

- **Authorization:** B should only release photos if authorized by A via `idp`.
- **Password secrecy:** A's password `pw(A,idp)` must not be revealed to P, B, or the intruder.
- **Photo confidentiality:** Photos should remain secret between B and P.
- **Authentication:** B must be able to verify `idp` signed the authorization token.

4 Week 2: Informal Design and Static Dolev–Yao Analysis

Section author: Kristófer Birgir Hjörleifsson

4.1 Informal Outline

The basic protocol flow:

1. A requests authorization from `idp` (encrypted with `idp`'s public key)
2. `idp` creates signed token authorizing P to access A's photos at B
3. A forwards token to P
4. P presents token to B (encrypted with B's public key)
5. B verifies token and sends photos to P (encrypted with P's public key)

4.2 First AnB Model (Simplified)

Initial simplification: A has a key pair (temporary, removed in Week 3). `idp` is lowercase—an honest constant that the intruder cannot instantiate.

Listing 1: Week 2 protocol

```
Actions:
  A -> idp: {A, P, B}(pk(idp))
  idp -> A: {A, P, B}(inv(pk(idp)))
  A -> P: {A, P, B}(inv(pk(idp)))
  P -> B: {{A, P, B}(inv(pk(idp)))}(pk(B))
  B -> P: {photos(A)}(pk(P))
```

4.3 OFMC Result

OFMC found an attack on the secrecy goal: the intruder plays A's role, requests authorization for itself, and manipulates the token flow so that B sends photos encrypted to a key the intruder controls. This is possible because A, B, P are all uppercase variables in AnB, meaning the intruder can instantiate them.

Key insight: Because `idp` is lowercase, the intruder *cannot* forge `idp`'s signature. The attack is about role manipulation, not signature forgery.

4.4 Special Task: Static Analysis (Passive Intruder)

Assumption: Intruder only eavesdrops (passive), does not inject messages.

What intruder observes:

- Message 1: `{A, P, B}(pk(idp))` – cannot decrypt (no `inv(pk(idp))`)
- Message 2: `{A, P, B}(inv(pk(idp)))` – can verify signature, learns A, P, B
- Message 3: Same signed token – learns nothing new
- Message 4: `{{token}}(pk(B))` – cannot decrypt outer layer
- Message 5: `{photos(A)}(pk(P))` – cannot decrypt

Intruder's knowledge (passive): Agent identities A, P, B (from signed token). **Cannot learn:** photos, password, or private keys.

Conclusion: Protocol is secure against passive eavesdropping. The active attacks require message injection and role manipulation.

5 Week 3: Identity Provider and Lazy Intruder Executability

Section author: Mikael Máni Eyfeld Clarke

5.1 Protocol Refinement

Key change: A no longer has a key pair. A authenticates to `idp` using shared password `pw(A,idp)`.

Listing 2: Week 3: Password authentication

```
A -> idp: {A, P, B, pw(A,idp)}(pk(idp))
```

The password is encrypted under `idp`'s public key, so the intruder cannot learn it. Password is NOT used as an encryption key—only as a credential.

5.2 Special Task: Lazy Intruder Method (P Executable by Intruder)

We show role P is executable by the intruder using constraint solving (lazy intruder).

Role P's messages:

- Receives from A: `token = {A, P, B}(inv(pk(idp)))`
- Sends to B: `{{A, P, B}(inv(pk(idp)))}(pk(B))`

Constraint solving (lecture style):
Initial intruder knowledge IK_0: all public names, `pk(B)`, `pk(P)`, `pk(idp)`, agent names.
After intercepting message 3 (A → P):

$$IK_1 = IK_0 \cup \{\texttt{\{A, P, B\}(inv(pk(idp)))}\}$$

The intruder must compose: `{{A, P, B}(inv(pk(idp)))}(pk(B))`
Resolution steps:

1. **Axiom:** `{A, P, B}(inv(pk(idp)))` $\in IK_1$ (intercepted token)
2. **Axiom:** `pk(B)` $\in IK_0$ (public key)
3. **Compose (encryption):** From steps 1–2, compose `{{A, P, B}(inv(pk(idp)))}(pk(B))`

Conclusion: Role P is executable by the intruder. All outgoing messages can be constructed from the intruder's knowledge via Axiom and Compose rules.

6 Week 4: Typing, Formats, and Public-Key Lookup

Section author: Kristófer Birgir Hjörleifsson

6.1 Type-Flaw Resistance via Format Tags

Type flaws occur when an intruder substitutes a message of one type where another is expected. We prevent this by adding constant format tags (`f1`–`f4`) as the first element of each message tuple, making every message structurally unique:

Listing 3: Week 4: Format-tagged messages

```
Msg 1: {f1, A, P, B, pw(A,idp)}(pk(idp)) % Auth request
Msg 2: {f2, A, P, B}(inv(pk(idp))) % Token
Msg 4: {f3, P, {f2, A, P, B}(inv(pk(idp)))}(pk(B)) % Photo request
Msg 5: {f4, B, photos(A)}(pk(P)) % Photo response
```

The tag `f1` marks auth requests, `f2` marks tokens, etc. Since each message begins with a distinct constant, no two different message types can be unified.

6.2 Public-Key Lookup Subprotocol

A separate protocol allows A to obtain public keys from `idp`:

Listing 4: Key lookup protocol

```
A -> idp: {f5, A, B, pw(A,idp)}(pk(idp))
idp -> A: {f5, A, B, pk(B)}(inv(pk(idp)))
```

6.3 Special Task: Type-Flaw Resistance Proof

Definition: A protocol is type-flaw resistant if for any two non-variable messages M_1, M_2 in the set of sub-messages of the protocol (SMP), if they have a unifier σ, then $\text{type}(M_1) = \text{type}(M_2)$.

Analysis of message types:

Message	Leading tag	Arity
Msg 1	f1	5 (f1, Agent, Agent, Agent, Password)
Msg 2	f2	4 (f2, Agent, Agent, Agent)
Msg 4 outer	f3	3 (f3, Agent, SignedToken)
Msg 5	f4	3 (f4, Agent, Data)

Unification check:

- No two messages share the same leading tag, so no unifier exists between different message types.
- Even ignoring tags: Msg 1 has 5 components, Msg 2 has 4, etc.—different arities prevent unification.
- The signed token in message 2 and the inner token carried inside message 4 do unify, but this is benign because they are the *same* tagged message type `f2`. Type-flaw resistance only requires that messages of *different* types cannot unify.

Conclusion: Protocol is type-flaw resistant.

7 Week 5: Channels (TLS-like Pseudonymous Channels)

Section author: Mikael Máni Eyfeld Clarke

7.1 Channel Notation

Notation	Meaning
`A -> B: M`	Insecure (Dolev-Yao)
`A *-> B: M`	Confidential only
`A ->* B: M`	Authenticated only
`A *->* B: M`	Secure channel (mutual TLS)
`[A] *->* B: M`	A pseudonymous, B authenticated (TLS server-auth)
`B *->* [A]: M`	Reply to pseudonymous A

7.2 Pseudonymous Channel Variant

Listing 5: Week 5: Pseudonymous TLS channels

```
[A] *->* idp: f1, A, P, B, pw(A,idp)
idp *->* [A]: {f2, A, P, B}(inv(pk(idp)))
[A] *->* P: {f2, A, P, B}(inv(pk(idp)))
P *->* B: f3, P, {f2, A, P, B}(inv(pk(idp)))
B *->* P: f4, B, photos(A)
```

The `[A]` notation means A is *pseudonymous*: the server authenticates itself (has a certificate) but does not know the client's identity. This models TLS with server-side authentication only. A authenticates at the application level via `pw(A,idp)`.

7.3 Special Task: What Was Replaced

Original	Channel Version	Replaced?
`{M}(pk(idp))`	`[A] *->* idp: M`	Yes
`{M}(pk(B))`	`P *->* B: M`	Yes
`{M}(pk(P))`	`B *->* P: M`	Yes
`{M}(inv(pk(idp)))`	Cannot replace	**No**

Why the signature cannot be replaced: B needs to verify `idp`'s approval, but B has no direct TLS channel to `idp`. The token travels through A and P, so B must verify the signature cryptographically.

8 Week 6: Privacy and Guessable Password

Section author: Kristófer Birgir Hjörleifsson

8.1 Guessable Secrets

A guessable secret (like a password) can be subject to offline dictionary attacks. The intruder captures data containing or derived from the password and tries verifying guesses offline.

8.2 Security with Guessable Password

Our protocol protects against guessing attacks because:

- Password is sent over a pseudonymous secure channel (`[A] *->* idp`)
- Intruder never sees any derivation of the password on the wire
- No offline verification possible

OFMC Goal: `pw(A,idp) guessable secret between A, idp`
Result: No guessing attack found. The only attack is on photo secrecy (intruder plays A), not on the password.

8.3 Special Task: Insecure Variant Demonstration

We created an insecure variant using hash-based challenge-response over an insecure channel. This is a proper example of offline guessing vulnerability (unlike public-key encryption, which does *not* allow the intruder to verify guesses):

Listing 6: Insecure version: hash-based challenge-response

```
A -> idp: A, P, B
idp -> A: NB
A -> idp: h(pw(A,idp), NB)
```

The intruder observes `NB` and `h(pw(A,idp), NB)` on the wire and can verify password guesses by recomputing the hash.
OFMC Result: `ATTACK_FOUND (guesswhat)`

Listing 7: Guessing attack trace

```
(A,1) -> i: A, P, B
i -> (A,1): NB
(A,1) -> i: h(pw(A,idp), NB)
i can produce secret h(guessPW, NB)
```

Conclusion: Public-key encryption (`{pw}(pk(idp))`) does *not* allow offline guessing because the intruder cannot decrypt without `inv(pk(idp))`. Hash-based challenge-response *does* allow it because the intruder can recompute `h(guess, NB)` and compare. Secure channels prevent both.

9 Final Protocol (AnB Specification Summary)

Section author: Mikael Máni Eyfeld Clarke

9.1 Final Message Flow

Listing 8: Final protocol (photo_auth_final.AnB)

```
[A] *->* idp: f1, A, P, B, pw(A,idp)
idp *->* [A]: {f2, A, P, B}(inv(pk(idp)))
[A] *->* P: {f2, A, P, B}(inv(pk(idp)))
P *->* B: f3, P, {f2, A, P, B}(inv(pk(idp)))
B *->* P: f4, B, photos(A)
```

Step-by-step:

1. A authenticates to `idp` with password over pseudonymous TLS

2. `idp` creates signed authorization token (format tag `f2`)
3. A forwards token to P over pseudonymous TLS
4. P presents token to B with mutual TLS (format tag `f3`)
5. B verifies token signature and sends photos to P (format tag `f4`)

9.2 Initial Knowledge

Agent	Knowledge
A	A, B, P, idp, pw(A,idp), f1, f2, f3, f4
B	A, B, P, idp, pk(B), pk(idp), inv(pk(B)), photos(A), f1, f2, f3, f4
P	A, B, P, idp, pk(B), pk(P), pk(idp), inv(pk(P)), f1, f2, f3, f4
idp	A, B, P, idp, pk(B), pk(P), pk(idp), inv(pk(idp)), pw(A,idp), f1, f2, f3, f4

9.3 Goals

- `B authenticates idp on f2, A, P, B` – B verifies `idp` signed the token
- `photos(A) secret between B, P` – Photos only known to B and P
- `pw(A,idp) guessable secret between A, idp` – Password protected

9.4 Verification Results

OFMC Result: ATTACK_FOUND (secrets)

OFMC reports a secrecy violation because the intruder can instantiate the user role A with itself (A is an uppercase variable). In that case, `idp` legitimately issues a token for the intruder's own identity, and the intruder eventually learns `photos(i)`—i.e. its own resource. This is not a meaningful break of the intended authorization property for honest users; it is a modeling artifact of variable role instantiation in the Dolev–Yao model.

Important: With `idp` as a lowercase constant, the intruder cannot forge `idp`'s signature or impersonate the identity provider. The attack is purely about the intruder playing a legitimate user role.

Password security: The guessable-password goal `pw(A,idp) guessable secret between A, idp` is *not* violated—this is the important Week 6 privacy result.

10 Assumptions, Simplifications, and Limitations

Section author: Kristófer Birgir Hjörleifsson

10.1 Assumptions

- **idp is honest:** Modeled as a lowercase constant in AnB. The intruder cannot instantiate the `idp` role.
- **Secure channels abstract TLS:** We assume TLS provides confidentiality and authentication as modeled by the channel notation.
- **Keys are pre-distributed:** Servers know each other's public keys (or obtain them via the key lookup subprotocol).

10.2 Simplifications

- **Photos as abstract secret:** Modeled as `photos(A)` rather than actual data.
- **No scope/folder:** Token authorizes access to all photos, not a specific subset.
- **No timestamps/nonces:** Token has no expiration or freshness mechanism.
- **Single session:** OFMC verifies a bounded number of sessions.

10.3 Limitations

- **Intruder can play A:** Since A, B, P are uppercase variables, the intruder can instantiate any of these roles. This leads to the photo secrecy attack where the intruder requests its own photos—a modeling artifact, not a real authorization break.
- **No revocation:** Once issued, tokens cannot be revoked.
- **idp single point of failure:** If `idp` were compromised (not modeled), all security guarantees fail.
- **No token freshness:** The token contains no nonce or timestamp, so replay of the token is possible within the OFMC bounded-session model.

11 Conclusion

Section author: Mikael Máni Eyfeld Clarke

We designed an open authorization protocol allowing user A to authorize printing service P to access photos at server B, using identity provider `idp` for authentication.

Key design decisions:

- `idp` as lowercase constant prevents the intruder from impersonating the identity provider
- Format tags (`f1`–`f4`) prevent type-flaw attacks
- Pseudonymous channels (`[A] *->*`) model TLS with server-side authentication
- Hash-based challenge-response is vulnerable to offline guessing; secure channels are not

Remaining attack: OFMC reports ATTACK_FOUND (secrets) because the intruder can instantiate A with itself, causing `idp` to issue a legitimate token for the intruder's own identity. The intruder then learns `photos(i)`—its own resource. This is inherent to the Dolev–Yao model: any uppercase variable role can be played by the intruder, and no secrecy goal formulation can prevent it. It is not a meaningful break of honest-user authorization. The guessable-password goal holds, which is the key Week 6 privacy result.

Final protocol provides:

- Authentication of `idp`'s authorization via signed tokens
- Confidentiality of photos between B and P (modulo the inherent Dolev–Yao modeling artifact)
- Protection of guessable password via pseudonymous TLS channels

A Submission Map (Where to Find Things)

Section author: Kristófer Birgir Hjörleifsson

- Lab logbook: `logbook/logbook.md`
- AnB protocol versions:
  - `anb/week2_v1.AnB` – Basic protocol
  - `anb/week3_v1.AnB` – Password authentication
  - `anb/week4_v1.AnB` – Type-flaw resistant
  - `anb/week5_v1.AnB` – Pseudonymous TLS channels
  - `anb/week5_v1_tls.AnB` – Crypto comparison (no channels)
  - `anb/week6_v1.AnB` – Guessable password
  - `anb/week6_insecure.AnB` – Insecure variant (demo)
- Final AnB files:
  - `anb/final/photo_auth_final.AnB` – Final protocol
  - `anb/final/key_lookup.AnB` – Key lookup subprotocol